CS 216: Bitcoin Transaction Lab

Team Name	HighOnByte
Course	CS 216
Assignment	2nd Programming Assignment
Deadline	10th March 2026

Team Members

Name	Roll Number
Aastha	240004001
Bhavika Jaiswal	240001017
Sanskriti Jain	240001064
Suhani	240001077

GitHub Repository:
https://github.com/Sanskriti-hello/bitcoin-transaction-lab

Contents

1 Theoretical Background **2**
1.1 Bitcoin Script . . . 2
1.2 P2PKH – Pay to Public Key Hash (Legacy) . . . 2
1.3 P2SH-P2WPKH – SegWit wrapped in P2SH . . . 2

2 Workflow Overview **4**

3 Part 1: Legacy P2PKH Transactions **5**
3.1 Bitcoin Core Configuration . . . 5
3.2 Generated Legacy Addresses . . . 5
3.3 Transaction Workflow: A to B to C . . . 5
3.4 Decoded Scripts – Transaction A to B . . . 5
3.5 Decoded Scripts – Transaction B to C . . . 6
3.6 Transaction Sizes – P2PKH . . . 6

4 Part 1: btcdeb Script Execution (P2PKH) **7**
4.1 Command Used . . . 7
4.2 Initial Script Load – 8 Opcodes . . . 7
4.3 Step-by-Step Execution . . . 7
4.4 Final Result . . . 9

5 Part 2: P2SH-P2WPKH SegWit Transactions **10**
5.1 Generated SegWit Addresses . . . 10
5.2 Transaction Workflow: A' to B' to C' . . . 10
5.3 Decoded Scripts – Transaction A' to B' . . . 10
5.4 Decoded Scripts – Transaction B' to C' . . . 10
5.5 Transaction Sizes – SegWit . . . 11

6 Part 2: btcdeb Script Execution (SegWit) **12**
6.1 Command Used . . . 12
6.2 Initial Load – Witness Stack Size 2 . . . 12
6.3 Step-by-Step Execution . . . 12
6.4 Final Result . . . 13

7 Part 3: Comparative Analysis **14**
7.1 Transaction Size Comparison . . . 14
7.2 Script Structure Comparison . . . 14
7.3 Weight Calculation – Worked Example . . . 14
7.4 Why SegWit Transactions Are Smaller . . . 14

8 Conclusion **16**

1 Theoretical Background

1.1 Bitcoin Script

Bitcoin Script is a simple, stack-based programming language used to define the conditions under which Bitcoin can be spent. It has no loops and is not Turing-complete, making it simple and safe to execute. Transaction validation works by combining two scripts:

- **scriptSig** (unlocking script) – provided by the spender in the input
- **scriptPubKey** (locking script) – stored in the UTXO being spent

They are concatenated and executed together. The transaction is valid only if the final stack value is `TRUE (0x01)`.

1.2 P2PKH – Pay to Public Key Hash (Legacy)

P2PKH is the classic Bitcoin address format. In regtest mode, addresses start with `m` or `n`.

Address derivation:

```
Private Key -> Public Key -> SHA256 -> RIPEMD160 -> Bitcoin Address
```

Challenge Script (scriptPubKey) – 25 bytes

```
OP_DUP OP_HASH160 <pubKeyHash_20bytes> OP_EQUALVERIFY OP_CHECKSIG
hex prefix: 76a914...88ac
```

Response Script (scriptSig) – 106 bytes

```
<DER_signature ~71bytes> <compressed_pubkey_33bytes>
```

Stack Execution Trace

Step	Operation	Stack State
0	Push signature (from scriptSig)	[sig]
1	Push pubkey (from scriptSig)	[sig, pubKey]
2	OP_DUP	[sig, pubKey, pubKey]
3	OP_HASH160	[sig, pubKey, hash160(pubKey)]
4	Push expected hash (from scriptPubKey)	[sig, pubKey, computed, expected]
5	OP_EQUALVERIFY – must match	[sig, pubKey]
6	OP_CHECKSIG – verify ECDSA sig	[TRUE / 0x01]

1.3 P2SH-P2WPKH – SegWit wrapped in P2SH

SegWit (BIP 141) separates signature data into a **witness** field. In P2SH-P2WPKH, a native SegWit script is wrapped inside a P2SH address for backward compatibility. Addresses start with `2` in regtest.

scriptPubKey (on-chain) – 23 bytes

```
OP_HASH160 <hash160(redeemScript)> OP_EQUAL
hex prefix: a914...87
```

redeemScript (revealed when spending) – 22 bytes

```
OP_0 <hash160(publicKey)>
hex: 0014<20bytes>
```

scriptSig – 23 bytes only

```
<serialized redeemScript>   <- only the redeem script, NOT the signature
```

Witness (segregated, weight discounted)

```
[0]: <DER_signature ~71bytes>
[1]: <compressed_pubkey 33bytes>
```

Weight Calculation

```
weight = (non-witness bytes x 4) + (witness bytes x 1)
vbytes = ceil(weight / 4)
Witness bytes get 75% discount compared to regular bytes.
```

2 Workflow Overview

```
Bitcoin Transaction Lab Workflow
--------------------------------

LEGACY (P2PKH)

Funding TX
    |
    v
Address A  (1 BTC, P2PKH)
    |
    |  TX1: A -> B
    v
Address B  (UTXO)
    |
    |  TX2: B -> C
    v
Address C  (Final UTXO)


SEGWIT (P2SH-P2WPKH)

Funding TX
    |
    v
Address A' (1 BTC, SegWit)
    |
    |  TX1: A' -> B'
    v
Address B' (UTXO)
    |
    |  TX2: B' -> C'
    v
Address C' (Final UTXO)


Comparison
P2PKH  -> 191 vbytes
SegWit -> 134 vbytes
Savings ~ 30%
```

3 Part 1: Legacy P2PKH Transactions

3.1 Bitcoin Core Configuration

`bitcoind` was run in regtest mode with the following `bitcoin.conf`:

```
regtest=1
server=1
txindex=1

[regtest]
rpcuser=student
rpcpassword=cs216bitcoin
rpcport=18443
rpcbind=127.0.0.1
rpcallowip=127.0.0.1
paytxfee=0.0001
fallbackfee=0.0002
mintxfee=0.00001
txconfirmtarget=6
```

3.2 Generated Legacy Addresses

Three legacy (P2PKH) addresses were generated using `getnewaddress "label" "legacy"`:

Label	Address
Address A	`mgdUJzJhqrZo9vKT3B2AcrZ81bngpJKrN6`
Address B	`mtJpEisXAjwimoBzEKVGBYFqr8Uot4P4PV`
Address C	`n4d6oYbGNvWSo2LsbsbtwCp8mknwnMW74S`

3.3 Transaction Workflow: A to B to C

Address A was funded with 1.0 BTC. The UTXO created at A was used as input to transaction A to B. The UTXO at B was then used as input to B to C, forming a verified UTXO chain.

Funding TXID (coins sent to A)

```
1df07c77d16e8f3ec642867864e37617a1febc3902426b44ba276152c7652afc
```

TXID: Transaction A to B

```
fafa0abb7b9126b6fae7bba7d0d24174491726a214ec4a00ca73f65396ab354c
```

TXID: Transaction B to C

```
d532428dc63c82f4b6a5c057e72c9e93ed08384f9ab13e41e02efffd63e2e8c7
```

Chain verification: The UTXO at address B was confirmed to have txid equal to the A to B transaction, verified programmatically using `assert utxo_b["txid"] == txid_ab`.

3.4 Decoded Scripts – Transaction A to B

Challenge Script (scriptPubKey) for Address B

```
type: pubkeyhash
asm:  OP_DUP OP_HASH160 8c4deaea5fa0ca2798347033d69abb0051af9f24
      OP_EQUALVERIFY OP_CHECKSIG
hex:  76a9148c4deaea5fa0ca2798347033d69abb0051af9f2488ac
size: 25 bytes
```

This script **locks** the output to address B. To spend it, the spender must provide a public key whose hash160 equals `8c4deaea5fa0ca2798347033d69abb0051af9f24` AND a valid ECDSA signature.

3.5 Decoded Scripts – Transaction B to C

Challenge Script (scriptPubKey) for Address C

```
type: pubkeyhash
asm:  OP_DUP OP_HASH160 fd747dc6bafedb20c308e28ffb034d778cbe0d0a
      OP_EQUALVERIFY OP_CHECKSIG
hex:  76a914fd747dc6bafedb20c308e28ffb034d778cbe0d0a88ac
size: 25 bytes
```

Response Script (scriptSig) – unlocking B's UTXO

```
asm:
  3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5af9ae721f3
  02204d634e0e14e38ee20f377b8b2f774ceb12c72f0e23f5e1d313749cb29a352911
  [ALL] 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5

hex:
  473044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5af9ae721f3
  02204d634e0e14e38ee20f377b8b2f774ceb12c72f0e23f5e1d313749cb29a352911
  0121033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5

size: 106 bytes
  - DER signature: ~71 bytes (includes SIGHASH_ALL byte)
  - Compressed public key: 33 bytes
  - Push data opcodes: 2 bytes
```

Challenge-Response Match Verification

Challenge requires	Response provides
hash160(pubKey) == 8c4deaea...	pubKey = 033504f7... whose hash160 = 8c4deaea...
valid ECDSA signature	sig = 3044022055ee... verified against pubKey

3.6 Transaction Sizes – P2PKH

Transaction	Size (bytes)	vSize (vbytes)	Weight (WU)
A to B	191	191	764
B to C	191	191	764

For P2PKH: size = vsize = weight/4 because there is no witness data. All bytes counted at weight x4.

4 Part 1: btcdeb Script Execution (P2PKH)

4.1 Command Used

```
btcdeb --tx=<signed_hex_B_to_C> --txin=<signed_hex_A_to_B>
```

btcdeb loads the spending transaction (B to C) and the UTXO being spent (A to B output), reconstructs the full script execution, and steps through each opcode showing stack state.

4.2 Initial Script Load – 8 Opcodes

```
(venv) sunny@DESKTOP-FVSQ52O:~/bitcoin_lab$ btcdeb --tx=02000000014c35ab9653f673ca004aec14a22617497441d2d0a7bbe7fab626917bbb0afafa000000006a473044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5af9ae72
1f302204d634e0e14e38ee20f377b8b2f774ceb12c72f0e23f5e1d313749cb29a35291101213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5fdffffff01e092f50500000001976a914fd747dc6bafedb20c308e28ffb034d778cbe
0d0a88ac00000000 --txin=0200000001fc2a65c7526127ba446b420239bcfea11776e364788642c63e8f6ed1777cf01d000000006a4730440220226ed977aa878b65c601fde9bd29856ac0218296fbd25d93df85e656ae4942300220260d0b7452c3a08a73e3798e73
4d1d153c57f168f0d16829edb4a33c312105c9b012103e0c96337fa816659872b6384f3ae65e98c7649b9d8216d80955046c83e376522fdffffff01f0b9f50500000001976a9148c4deaea5fa0ca2798347033d69abb0051af9f2488ac00000000
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
input tx index = 0; tx input vout = 0; value = 99990000
got witness stack of size 0
8 op script loaded. type `help` for usage information
script                                                          |  stack
----------------------------------------------------------------+--------
3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5a... |
033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5 |
<<< scriptPubKey >>>                                            |
OP_DUP                                                          |
OP_HASH160                                                      |
8c4deaea5fa0ca2798347033d69abb0051af9f24                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
#0000 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5af9ae721f302204d634e0e14e38ee20f377b8b2f774ceb12c72f0e23f5e1d313749cb29a35291101
```

Figure 1: btcdeb: Initial state – 8 op script loaded, witness stack size 0 (P2PKH/non-segwit). Both signature and pubkey visible in script column, scriptPubKey opcodes listed below.

4.3 Step-by-Step Execution

```
btcdeb> step
                <> PUSH stack 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5af9ae721f302204d634e0e14e38ee20f377b8b2f774ceb12c72f0e23f5e1d313749cb29a35291101
script                                                          |                                                                  stack
----------------------------------------------------------------+------------------------------------------------------------------
033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5 | 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5a...
<<< scriptPubKey >>>                                            |
OP_DUP                                                          |
OP_HASH160                                                      |
8c4deaea5fa0ca2798347033d69abb0051af9f24                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
#0001 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
```

Figure 2: Step 0: Signature pushed onto stack from scriptSig. Stack: [sig]

```
btcdeb> step
                <> PUSH stack 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
script                                                          |                                                                  stack
----------------------------------------------------------------+------------------------------------------------------------------
<<< scriptPubKey >>>                                            | 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
OP_DUP                                                          | 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5a...
OP_HASH160                                                      |
8c4deaea5fa0ca2798347033d69abb0051af9f24                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
<<< scriptPubKey >>>
```

Figure 3: Step 1: Public key `033504f752d213985732db96...` pushed. Stack: [pubKey, sig]

```
btcdeb> step
script                                                          |                                                                  stack
----------------------------------------------------------------+------------------------------------------------------------------
OP_DUP                                                          | 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
OP_HASH160                                                      | 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5a...
8c4deaea5fa0ca2798347033d69abb0051af9f24                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
#0003 OP_DUP
```

Figure 4: Step 2 (OP_DUP): Top of stack duplicated. Stack: [pubKey, pubKey, sig]

```
btcdeb> step
                <> PUSH stack 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
OP_HASH160                                                      | 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
8c4deaea5fa0ca2798347033d69abb0051af9f24                        | 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
OP_EQUALVERIFY                                                  | 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5a...
OP_CHECKSIG                                                     |
#0004 OP_HASH160
```

Figure 5: Step 3 (OP_HASH160): Top pubKey hashed. Stack: [hash160(pubKey), pubKey, sig]

```
btcdeb> step
                <> POP  stack
                <> PUSH stack 8c4deaea5fa0ca2798347033d69abb0051af9f24
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
8c4deaea5fa0ca2798347033d69abb0051af9f24                        |                                  8c4deaea5fa0ca2798347033d69abb0051af9f24
OP_EQUALVERIFY                                                  | 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
OP_CHECKSIG                                                     | 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5a...
#0005 8c4deaea5fa0ca2798347033d69abb0051af9f24
```

Figure 6: Step 4: Expected hash `8c4deaea5fa0ca2798347033d69abb0051af9f24` pushed. Stack: [expected, computed, pubKey, sig]

```
btcdeb> step
                <> PUSH stack 8c4deaea5fa0ca2798347033d69abb0051af9f24
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
OP_EQUALVERIFY                                                  |                                  8c4deaea5fa0ca2798347033d69abb0051af9f24
OP_CHECKSIG                                                     |                                  8c4deaea5fa0ca2798347033d69abb0051af9f24
                                                                | 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
                                                                | 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5a...
#0006 OP_EQUALVERIFY
```

Figure 7: Step 5 (OP_EQUALVERIFY): Both hashes compared. They match – both popped. Stack: [pubKey, sig]

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
OP_CHECKSIG                                                     | 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
                                                                | 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5a...
#0007 OP_CHECKSIG
```

Figure 8: Step 6: OP_CHECKSIG about to execute – pubKey and sig on stack ready for ECDSA verification

```
btcdeb> step
EvalChecksig() sigversion=0
Eval Checksig Pre-Tapscript
GenericTransactionSignatureChecker::CheckECDSASignature(71 len sig, 33 len pubkey, sigversion=0)
  sig         = 3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5af9ae721f302204d634e0e14e38ee20f377b8b2f774ceb12c72f0e23f5e1d313749cb29a35291101
  pub key     = 033504f752d213985732db96c47aba8beaeb1e563839965fd44cdfb5562ec94af5
  script code = 76a9148c4deaea5fa0ca2798347033d69abb0051af9f2488ac
  hash type   = 01 (SIGHASH_ALL)
SignatureHash(nIn=0, nHashType=01, amount=99990000)
- sigversion = SIGVERSION_BASE (non-segwit style)
 << txTo.vin[nInput=0].prevout = COutPoint(fafa0abb7b, 0)
(SerializeScriptCode)
 << scriptCode.size()=25 - nCodeSeparators=0
 << script:76a9148c4deaea5fa0ca2798347033d69abb0051af9f2488ac
 << txTo.vin[nInput].nSequence = 4294967293 [0xfffffffd]
  sighash     = 074dfcaffb73bf90893ca01d794243850ff34c2cd6e471b222cf2987c41fde77
  pubkey.VerifyECDSASignature(sig=3044022055ee689066e5bd7ea7d5ec7091d1e4484a1a1ef91d0fc0a13569d5af9ae721f302204d634e0e14e38ee20f377b8b2f774ceb12c72f0e23f5e1d313749cb29a352911, sighash=074dfcaffb73bf90893ca01d794
243850ff34c2cd6e471b222cf2987c41fde77):
  result: success
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
                                                                |                                                                  01
```

Figure 9: Step 7 (OP_CHECKSIG): ECDSA verification complete. sigversion=SIGVERSION_BASE. result: success. Final stack: [01] = TRUE

4.4 Final Result

OP_CHECKSIG result: success
sigversion = SIGVERSION_BASE (non-segwit / legacy style)
Final stack: `[01]` = TRUE
Transaction B to C is VALID

5 Part 2: P2SH-P2WPKH SegWit Transactions

5.1 Generated SegWit Addresses

Three P2SH-SegWit addresses were generated using `getnewaddress "label" "p2sh-segwit"`:

Label	Address
Address A'	2MwDLiQDFFdMTGNcoZNFJdzbWyZ5cGuSHwB
Address B'	2Mws7da6CgS6HDuzBHtvixvz2awkQ5haTRG
Address C'	2N8gyrug6h3uMr51cFdR91VkeJ5tBWv6vC8

5.2 Transaction Workflow: A' to B' to C'

Funding TXID (coins sent to A')

```
a5108f50c7e5d7927b80b6d62e2689e7c9d73791a7c910295d77ef44c52ec6d1
```

TXID: Transaction A' to B'

```
b92b2ef1b3f4daac1b6bc6da28cf5b46c6d1333f7568308b8dade4e2521bc7c0
```

TXID: Transaction B' to C'

```
b536bb250cb504ff206fd517a41b9b8595868dee4083942bd699e87a58a61564
```

5.3 Decoded Scripts – Transaction A' to B'

Challenge Script (scriptPubKey) for Address B'

```
type: scripthash
asm:  OP_HASH160 32aa2bcdb8f5ede4a60b2f300f919aaaf5745796 OP_EQUAL
hex:  a91432aa2bcdb8f5ede4a60b2f300f919aaaf574579687
size: 23 bytes
```

This P2SH script commits only to the hash of the redeemScript. The actual redeemScript is revealed only when spending.

5.4 Decoded Scripts – Transaction B' to C'

Challenge Script (scriptPubKey) for Address C'

```
type: scripthash
asm:  OP_HASH160 a968e8b4c7a4d02a4e2d5437737c617bf0d75bd0 OP_EQUAL
hex:  a914a968e8b4c7a4d02a4e2d5437737c617bf0d75bd087
size: 23 bytes
```

Response Script (scriptSig) – spending B' UTXO

```
asm:  0014ffb9dcac2cca1e4ea378498d4f1beda758b771e3
hex:  160014ffb9dcac2cca1e4ea378498d4f1beda758b771e3
size: 23 bytes only

Decoded redeemScript: OP_0 <ffb9dcac2cca1e4ea378498d4f1beda758b771e3>
  = OP_0 <hash160(publicKey)>  <- P2WPKH witness program (22 bytes)
```

Key difference from P2PKH: scriptSig contains ONLY the 23-byte redeemScript. Signature and pubkey are segregated in the witness field.

Witness Data (segregated)

```
[0] signature (71 bytes, weight x1):
    3044022075f0860138569067736ab12bec4ef0fdd18fedb3dfddbb0dfb54e6ee58bd5962
    0220779c1ccacf103adc427f52626212545b68a3f214bedba0663a6dfd270eb81ca001

[1] public key (33 bytes, weight x1):
    0245085c3e415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c
```

Challenge-Response Match

Challenge (scriptPubKey)	Response (scriptSig + witness)
OP_HASH160 32aa2bcdb... OP_EQUAL	scriptSig reveals redeemScript hash160(redeemScript) must == 32aa2bcdb... witness[1] pubkey hashed must match OP_0 program witness[0] signature verified against pubkey

5.5 Transaction Sizes – SegWit

Transaction	Size (bytes)	vSize (vbytes)	Weight (WU)
A' to B'	215	134	533
B' to C'	215	134	533

Raw size (215) is larger than P2PKH (191) because witness data is included. But vsize (134) is much smaller than P2PKH (191) due to the 75% witness discount.

6 Part 2: btcdeb Script Execution (SegWit)

6.1 Command Used

```
btcdeb --tx=<signed_hex_B_prime_to_C_prime>
       --txin=<signed_hex_A_prime_to_B_prime>
```

6.2 Initial Load – Witness Stack Size 2

```
(venv) sunny@DESKTOP-FVSQ52O:~/bitcoin_lab$ btcdeb --tx=02000000000101c0c71b52e2e4ad8d8b3068753f33d1c6465bcf28dac66b1bacdaf4b3f12e2bb900000000017160014ffb9dcac2cca1e4ea378498d4f1beda758b771e3fdffffff01e092f505000
00000017a914a968e8b4c7a4d02a4e2d5437737c617bf0d75bd08702473044022075f08601385690677 36ab12bec4ef0fdd18fedb3dfddbb0dfb54e6ee58bd59620220779c1ccacf103adc427f52626212545b68a3f214bedba0663a6dfd270eb81ca001210245085c3e
415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c00000000 --txin=02000000000101d1c62ec544ef775d2910c9a79137d7c9e789262ed6b6807b92d7e5c7508f10a501000000017160014c1aa6e6da6fc63969307e4772b27507ef50a24f9fdfff
fff01f0b9f5050000000017a91432aa2bcdb8f5ede4a60b2f300f919aaaf5745796870247304022069698 70bc46ee27b0dcbe402646ef4597e2b5ab3f365212071a7fa29aeaf407e02205aa16b4c55ad09cb44f17614ea417397868c69718f17799dcfaa84b93b6a49
440121033013eff56cb0bc8c1a5e7156e2491c671d4ff0a87a016282485830 03a17900400000000
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
input tx index = 0; tx input vout = 0; value = 99990000
got witness stack of size 2
script sig non-empty; embedded P2SH (extracting payload)
hash source = 0014ffb9dcac2cca1e4ea378498d4f1beda758b771e3
22 bytes (P2WPKH)
valid script
- generating prevout hash from 1 ins
[+] COutPoint(b92b2ef1b3, 0)
note: there is a for-clarity preamble (use --verbose for details)
5 op script loaded. type `help` for usage information
script                                  |                                                                    stack
----------------------------------------+--------------------------------------------------------------------
OP_DUP                                  | 0245085c3e415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c
OP_HASH160                              | 3044022075f08601385690677 36ab12bec4ef0fdd18fedb3dfddbb0dfb54e6e...
ffb9dcac2cca1e4ea378498d4f1beda758b771e3 |
OP_EQUALVERIFY                          |
OP_CHECKSIG                             |
#0000 OP_DUP
```

Figure 10: btcdeb: SegWit initial load – got witness stack of size 2, embedded P2SH detected, P2WPKH (22 bytes) extracted, 5 op script loaded. Stack pre-populated with pubKey and sig from witness.

The startup shows: `script sig non-empty; embedded P2SH (extracting payload)`, `22 bytes (P2WPKH)`, `5 op script loaded`. The inner P2WPKH script runs with the witness data already on the stack.

6.3 Step-by-Step Execution

```
btcdeb> step
                <> PUSH stack 0245085c3e415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c
script                                  |                                                                    stack
----------------------------------------+--------------------------------------------------------------------
OP_HASH160                              | 0245085c3e415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c
ffb9dcac2cca1e4ea378498d4f1beda758b771e3 | 0245085c3e415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c
OP_EQUALVERIFY                          | 3044022075f08601385690677 36ab12bec4ef0fdd18fedb3dfddbb0dfb54e6e...
OP_CHECKSIG                             |
#0001 OP_HASH160
```

Figure 11: Step 0 (OP_DUP): Public key `0245085c3e415932ea3eff76...` duplicated. Stack: [pubKey, pubKey, sig]

```
btcdeb> step
                <> POP  stack
                <> PUSH stack ffb9dcac2cca1e4ea378498d4f1beda758b771e3
script                                  |                                                                    stack
----------------------------------------+--------------------------------------------------------------------
ffb9dcac2cca1e4ea378498d4f1beda758b771e3 |                                    ffb9dcac2cca1e4ea378498d4f1beda758b771e3
OP_EQUALVERIFY                          | 0245085c3e415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c
OP_CHECKSIG                             | 3044022075f08601385690677 36ab12bec4ef0fdd18fedb3dfddbb0dfb54e6e...
#0002 ffb9dcac2cca1e4ea378498d4f1beda758b771e3
```

Figure 12: Step 1 (OP_HASH160): Top pubKey hashed – `ffb9dcac2cca1e4ea378498d4f1beda758b771e3` computed and pushed

```
btcdeb> step
                <> PUSH stack ffb9dcac2cca1e4ea378498d4f1beda758b771e3
script                                  |                                                                   stack
----------------------------------------+-------------------------------------------------------------------
OP_EQUALVERIFY                          |                           ffb9dcac2cca1e4ea378498d4f1beda758b771e3
OP_CHECKSIG                             |                           ffb9dcac2cca1e4ea378498d4f1beda758b771e3
                                        | 0245085c3e415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c
                                        | 3044022075f0860138569067736ab12bec4ef0fdd18fedb3dfddbb0dfb54e6e...

#0003 OP_EQUALVERIFY
```

Figure 13: Step 2: Expected hash `ffb9dcac2cca1e4ea378498d4f1beda758b771e3` pushed for comparison. Both hashes now visible on stack.

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                  |                                                                   stack
----------------------------------------+-------------------------------------------------------------------
OP_CHECKSIG                             | 0245085c3e415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c
                                        | 3044022075f0860138569067736ab12bec4ef0fdd18fedb3dfddbb0dfb54e6e...

#0004 OP_CHECKSIG
```

Figure 14: Step 3 (OP_EQUALVERIFY): Both hashes match and verified – both popped. Stack: [pubKey, sig]

```
btcdeb> step
EvalChecksig() sigversion=1
Eval Checksig Pre-Tapscript
GenericTransactionSignatureChecker::CheckECDSASignature(71 len sig, 33 len pubkey, sigversion=1)
  sig         = 3044022075f0860138569067736ab12bec4ef0fdd18fedb3dfddbb0dfb54e6ee58bd59620220779c1ccacf103adc427f52626212545b68a3f214bedba0663a6dfd270eb81ca001
  pub key     = 0245085c3e415932ea3eff76b3890388d13a03ff461882db7b6a95067a6289a77c
  script code = 76a914ffb9dcac2cca1e4ea378498d4f1beda758b771e388ac
  hash type   = 01 (SIGHASH_ALL)
SignatureHash(nIn=0, nHashType=01, amount=99990000)
* sigversion == SIGVERSION_WITNESS_V0
  sighash     = b182232b20229642843c37dfd41da4fa744d8efa913cc9006a7d263526b2c0a0
  pubkey.VerifyECDSASignature(sig=3044022075f0860138569067736ab12bec4ef0fdd18fedb3dfddbb0dfb54e6ee58bd59620220779c1ccacf103adc427f52626212545b68a3f214bedba0663a6dfd270eb81ca0, sighash=b182232b20229642843c37dfd41
da4fa744d8efa913cc9006a7d263526b2c0a0):
  result: success
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                  |                                                                   stack
----------------------------------------+-------------------------------------------------------------------
                                        |                                                                     01
```

Figure 15: Step 4 (OP_CHECKSIG): sigversion=SIGVERSION_WITNESS_V0 (BIP 143). ECDSA verification complete. result: success. Final stack: [01] = TRUE

6.4 Final Result

OP_CHECKSIG result: success
sigversion = SIGVERSION_WITNESS_V0 (BIP 143 sighash)
Final stack: `[01]` = TRUE
Transaction B' to C' is VALID

Critical difference from P2PKH: sigversion was `SIGVERSION_WITNESS_V0`, meaning BIP 143 sighash algorithm was used, which explicitly commits to the input value (prevents amount manipulation). The witness data was pre-loaded onto the stack before execution, not pushed via scriptSig.

7 Part 3: Comparative Analysis

7.1 Transaction Size Comparison

All values measured from actual transactions broadcast on regtest:

Metric	P2PKH (B-C)	SegWit (B'-C')	Difference	Savings
Size (bytes)	191	215	+24	SegWit larger raw
vSize (vbytes)	191	134	-57	29.8% smaller
Weight (WU)	764	533	-231	30.2% smaller

Raw byte size of SegWit is larger because it physically includes witness data. However, vsize is significantly smaller because witness bytes are counted at only 1/4 weight.

7.2 Script Structure Comparison

Component	P2PKH (Legacy)	P2SH-P2WPKH (SegWit)
scriptPubKey type	pubkeyhash	scripthash
scriptPubKey size	25 bytes	23 bytes
scriptPubKey opcodes	OP_DUP OP_HASH160 hash OP_EQUALVERIFY OP_CHECKSIG	OP_HASH160 hash OP_EQUAL
scriptSig content	signature + pubkey	redeemScript only
scriptSig size	106 bytes	23 bytes
Witness	none	signature + pubkey
Witness size	0 bytes	104 bytes
sigversion	SIGVERSION_BASE	SIGVERSION_WITNESS_V0
Sighash algorithm	Legacy double SHA256	BIP 143 (commits to value)

7.3 Weight Calculation – Worked Example

P2PKH (191 bytes, no witness)

```
All bytes are non-witness:
weight = 191 x 4 = 764 WU
vbytes = 764 / 4 = 191 vbytes
```

P2SH-P2WPKH (215 bytes total, 533 WU measured)

```
Non-witness bytes: ~107 bytes (version, inputs without sig, outputs, locktime)
Witness bytes:     ~104 bytes (sig ~71B + pubkey 33B)

weight = (107 x 4) + (104 x 1)
       = 428 + 104 + 1 (+1 WU comes from marker/flag bytes)
       = 533 WU

vbytes = ceil(533 / 4) = 134 vbytes
```

7.4 Why SegWit Transactions Are Smaller

Reason 1: Witness Data Weight Discount

Bitcoin's weight unit system assigns different costs:

- **Non-witness data**: counted at weight **x4**
- **Witness data** (signatures, pubkeys): counted at weight **x1** (75% discount)

By moving signatures and pubkeys (104 bytes) to the witness field, their weight contribution drops by 75%.

Reason 2: scriptSig Weight Contribution

Part	P2PKH weight	SegWit weight
scriptSig (106 bytes vs 23 bytes)	106 x 4 = 424 WU	23 x 4 = 92 WU
witness (0 vs 104 bytes)	0 x 1 = 0 WU	104 x 1 = 104 WU
Total sig+pubkey contribution	**424 WU**	**196 WU**

Reason 3: Fee Benefit

```
Bitcoin fees = fee_rate (sat/vbyte) x vbytes

At 10 sat/vbyte:
  P2PKH:  191 vbytes x 10 = 1910 satoshis
  SegWit: 134 vbytes x 10 = 1340 satoshis
  Saving: 570 satoshis per transaction (~30%)
```

Reason 4: Block Throughput

Bitcoin block limit is 4,000,000 weight units. SegWit transactions use fewer weight units per transaction, allowing more transactions per block – increasing network throughput without a hard fork.

Reason 5: Transaction Malleability Fix

In P2PKH, signatures are inside scriptSig which is part of txid computation. A third party could alter the signature to produce a different txid – this is **transaction malleability**. In SegWit, signatures are in the witness field which is NOT part of txid, eliminating malleability and enabling the Lightning Network.

8 Conclusion

The experiment demonstrates how Bitcoin's stack-based scripting system validates transactions through a challenge-response mechanism. In legacy transactions the unlocking data (signature and public key) resides entirely in scriptSig, whereas SegWit moves this data into a segregated witness field. This design reduces transaction weight and fixes transaction malleability because the witness is excluded from the txid calculation.

Key findings:

- The UTXO chain was correctly formed: txid `fafa0abb7b...` (A to B) became input for B to C, and txid `b92b2ef1b3...` (A' to B') became input for B' to C'.
- P2PKH challenge scripts: 25 bytes, `OP_DUP OP_HASH160 ... OP_EQUALVERIFY OP_CHECKSIG`. SegWit challenge scripts: 23 bytes, `OP_HASH160 ... OP_EQUAL`.
- P2PKH scriptSig: 106 bytes (signature + pubkey). SegWit scriptSig: only 23 bytes (redeemScript); 104 bytes move to witness.
- btcdeb confirmed `result: success` for both P2PKH (sigversion=BASE) and SegWit (sigversion=WITNESS_V0).
- Real measured savings: 191 vbytes (P2PKH) vs 134 vbytes (SegWit) = 29.8% reduction in effective size.
- SegWit BIP 143 sighash commits to input value, preventing amount manipulation attacks.

Repository: https://github.com/Sanskriti-hello/bitcoin-transaction-lab